

Understanding Branches and Designing Branch Predictors for High-Performance Microprocessors

MARIUS EVERS AND TSE-YU YEH, MEMBER, IEEE

Invited Paper

Branch prediction is important in high-performance processors and its importance continues to grow. In the drive for higher execution frequencies, pipelines are lengthened and memory latencies are increased. This increases the cost of branch mispredictions.

In this paper, we describe some behavior patterns of branches. We believe that understanding the behavior of branches is helpful when designing fetch mechanisms for high-performance microprocessors. We also examine several current branch predictors and discuss how they work.

Finally, we look at some of the challenges that we are faced with when designing fetch mechanisms and predictors for future microprocessors and discuss some of the possible solutions.

Keywords—Branch prediction, speculation.

I. INTRODUCTION

Branch prediction is important in high-performance microprocessor design. There are two trends that are increasing the importance of branch prediction further: Processors are getting wider and pipelines are getting deeper. Increasing memory latencies may have effects similar to deepening pipelines. These trends make branch prediction critical, because they increase the relative cost of branch mispredictions. Furthermore, we can consider branch prediction as an enabling technology, since other algorithms such as sophisticated prefetch, runahead fetch engines, and out-of-order execution can be more effective if we stay on the correct path.

To illustrate the performance cost of branch mispredictions, consider a processor that can execute five instructions per cycle where branches are resolved in the tenth pipeline

stage. Because the younger instructions in the pipeline must be flushed on a misprediction, there will be $9 \times 5 = 45$ bubbles introduced into the pipeline for each misprediction. If every fifth instruction is a branch and the branch misprediction rate is 10% (the misprediction rate of a simple dynamic predictor), every 50th instruction is a mispredicted branch. For every 50 instructions fetched on the correct path, 45 instructions—almost half the fetch bandwidth—is lost. On the other hand, if the misprediction rate is 4%, every 125th instruction is a mispredicted branch. In this case, only $45/170 = 26\%$ of the fetch bandwidth is lost. Clearly, the branch prediction accuracy has a strong effect on performance.

Microprocessor designers need to understand branches. Branches define the path that a program must follow and therefore contain the information used by the fetch unit to feed the execution core. Simple characteristics such as branch frequency and taken rates help us design the fetch engine and decide which tradeoffs are appropriate. More complex characteristics, such as correlation, help us understand how to build a branch predictor that can predict well and how to improve branch prediction for specific classes of applications. We examine some of these branch characteristics in this paper.

Designers also need to know what prediction algorithms already exist. New advances are often improvements on existing schemes and for some uses, an existing scheme may be sufficient. We present what we consider to be the most fundamental algorithms and provide an evaluation of their performance for the SPECint95 benchmarks to give an indication of their abilities.

Finally, we look at what parts of the branch predictor design are likely to cause problems in future microprocessors. We examine a few of the critical loops in the fetch engine of a processor and discuss ways of achieving high prediction accuracy while still meeting the fundamental demands

Manuscript received January 4, 2001; revised June 12, 2001.

M. Evers is with Advanced Micro Devices, Sunnyvale, CA 94088 USA (e-mail: marius.evers@amd.com).

T.-Y. Yeh is with the Broadcom Corporation, Santa Clara, CA 95054 USA (e-mail: tyeh@broadcom.com).

Publisher Item Identifier S 0018-9219(01)09692-X.

Table 1
Description of Benchmarks and Their Input Sets

Benchmark	Description	Test Set	#Dynamic Instructions	#Dynamic Branches	#Dynamic Cond BR	#Static Cond BR
compress	Data compression	test.in*	103,015,025	17,069,558	10,661,855	310
gcc	GNU C compiler	jump.i	154,450,036	34,060,956	25,903,086	14,755
go	The go game	2stone9.in*	125,636,236	22,047,565	17,924,928	4,891
jpeg	Image compression	specmun.ppm*	206,802,135	24,147,330	20,441,307	1,179
m88ksim	Architectural simulator	dcrand.train.big	120,720,559	23,212,206	16,719,523	991
perl	Perl interpreter	scrabbl.pl*	78,148,849	16,030,598	10,570,887	1,670
vortex	Database	vortex.in*	231,997,610	43,171,027	33,853,896	6,385
xlisp	XLISP interpreter	train.lsp	187,724,756	44,758,353	26,422,064	512

*The input set is a modified version of one of the SPECint95 data sets.

of high-frequency processors. However, since each microprocessor design is unique, full solutions to the implementation issues cannot be given here.

II. SIMULATION METHODOLOGY

The experiments in this paper were conducted using a trace driven microarchitectural simulator for a load/store architecture using the instruction set of the Motorola MC 88000.

The metric used for comparing predictors is the branch misprediction rate. Although it is desirable to rate a predictor by the impact it has on the performance (such as IPC) of a processor, this cannot be done without selecting a particular processor implementation. Misprediction rate was chosen as it is most commonly used in the research literature. The prediction accuracy and, therefore, also the misprediction rate, has previously been shown to be strongly correlated to the performance of a processor [1].

All results presented are for the eight programs from the SPECint95 [2] suite. These benchmarks cover a diverse set of general purpose applications with varying characteristics, including some benchmarks with large footprints and many static conditional branches. Table 1 gives a short description of each of the benchmarks and lists the test data sets used. Different training data sets were used to generate the benchmark profiles for the experiments requiring profiling. All data sets are either the test, training, or reference data sets provided with the SPECint95 benchmarks or abbreviated versions of these. The benchmarks were always simulated until completion.

Table 1 also lists detailed information about the execution of the test input set for each of the benchmarks. The total number of instructions executed during the program run is listed in the fourth column. The number of branches executed is listed in the fifth column. The number of conditional branches is listed in the sixth column, and the number of static conditional branches that were executed at least once during the run of the program is listed in the final column.

III. BRANCH CHARACTERISTICS AND BEHAVIOR

In this section, we look at some fundamental characteristics of how a branch behaves and why. Due to the complexities of some aspects of branch behavior, certain issues cannot be effectively covered in a single paper. We have attempted

to convey a basic understanding of the issues and provide references for further investigation.

A. Branch Properties

Three basic branch properties are investigated in this section: type, frequency, and taken rates. The experiments are similar to those done in [3], but are reproduced here as the benchmarks and input data sets differ slightly. These properties are easily measured and are useful for understanding branch behavior.

Branches can be divided into conditional and unconditional branches. Based on the source of the target address, unconditional branches can be further divided into immediate branches, indirect branches, and returns. Immediate branches have the target address encoded in the branch instruction, indirect branches get their target address from a register, and returns get their target address from a link register or a stack. Technically, indirect branches and returns could also be conditional. However, most ISAs do not allow this. Fig. 1 shows the distribution of the branch types encountered when executing each of the SPECint95 benchmarks. The majority of the branches, 72% on average, are conditional, 17% are unconditional immediate, 10% are returns, and 1% are indirect. This highlights the importance of conditional branch prediction. Of the unconditional branches, immediate branches can be predicted accurately using a branch target buffer (BTB) and returns can be predicted accurately using a return address stack [4]. Target prediction for indirect branches was addressed in [5]. For conditional branches, both the target address and the direction must be predicted. Since conditional branches are generally immediate, the target address can be predicted accurately using a BTB. We will focus on predicting the direction of conditional branches here.

Fig. 2 shows the distribution of the execution frequencies of static conditional branches. Most static branches are only executed a few times during the run of a program. On average, 53% of all branches were executed 99 times or fewer. Only 11% of all branches were executed 10 000 times or more. Fig. 3 shows a similar distribution, but this time with each branch weighted by its execution frequency. This shows the representation of each of the categories in the dynamic instruction stream. The 53% of the branches that were executed 99 times or fewer make up only 0.2% of the branches in the dynamic instruction stream. The 11% of the branches

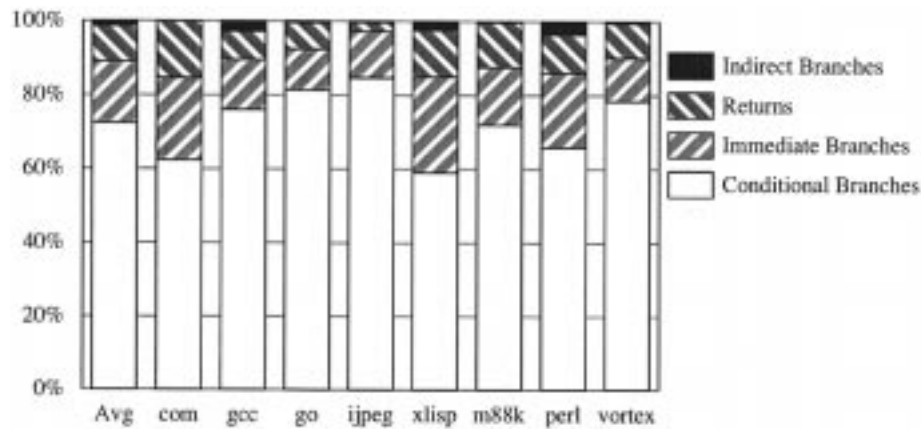


Fig. 1. Weighted distribution of branches by type.

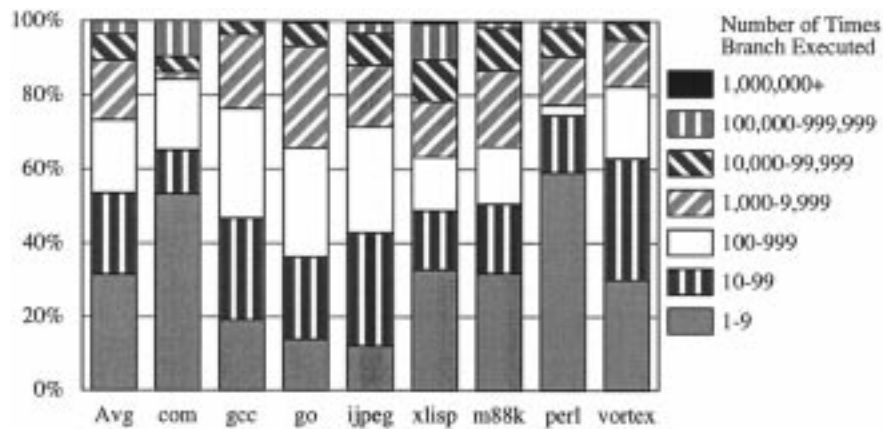


Fig. 2. Distribution of static branches by execution frequency.

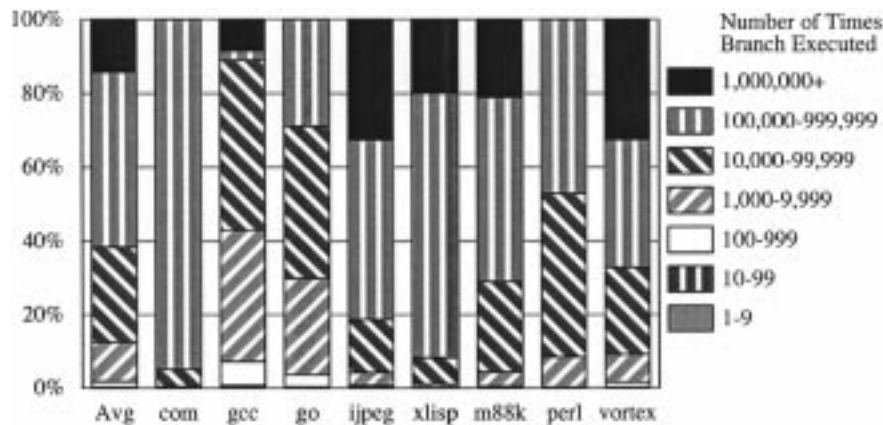


Fig. 3. Weighted distribution of static branches by execution frequency.

that were executed 10 000 times or more make up 87% of the branches in the dynamic instruction stream. This confirms the rule of thumb that 10% of the code is responsible for 90% of the execution of a program. This also emphasizes that frequently executed branches account for the vast majority of all branches that are executed. Even in the go and gcc benchmarks, which have a larger number of static branches, branches executed 99 times or fewer account for less than one percent of the branches in the dynamic instruction stream. Due to the dominance of frequently executed branches in the

instruction stream, it is very important to predict the frequent branches well.

Fig. 4 shows the distribution of the taken rates of static branches with each branch weighted by its execution frequency. The taken rate of a branch is the fraction of the time that branch was taken during the complete run of the program. Twenty-eight percent of all branches in the dynamic instruction stream were instances of static branches that were either always taken or never taken. Another 32% of the branches were less than 5% taken or more than 95%

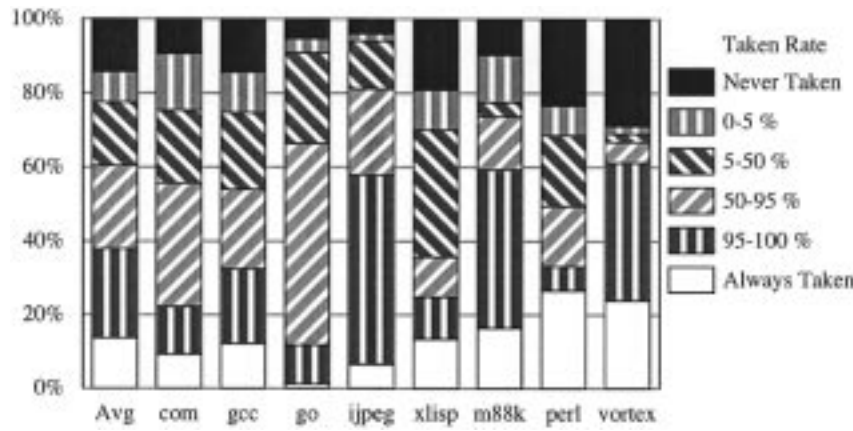


Fig. 4. Weighted distribution of static branches by taken rate.

taken. These 60% of branches are strongly biased in one direction and should be easy to predict. Forty percent of the branches have taken rates between 5% and 95%. It is for these branches that prediction is most challenging.

Profiling compilers can take advantage of knowledge about the dominant direction and predictability of a branch if we can determine this at profile time. To determine the stability of the taken rates when the input set changes, we compared the taken rate of each branch using a profiling set to the taken rates found with the testing set.

We found that 86% of the heavily biased branches (more than 95% biased) in the dynamic instruction stream have similar taken rates (difference of less than 0.5%) in the profiling and testing sets. However, for the branches with taken rates of 5%–95%, there was a 42% chance of the taken rate changing by more than 5%. Based on this, we see that profile data is fairly accurate for heavily biased branches, but less accurate for the remaining branches.

B. Looping and Other Systematic Behavior

It is possible to examine the branch patterns beyond looking only at simple taken rates. We saw in the previous section that 28% of all branches are always taken or always not taken. Many of the remaining branches follow repeating or other systematic patterns that can easily be detected.

One of the most common reasons for a branch to follow a repeating execution pattern is that it is the loop ending branch, or a branch inside the loop body. The pattern that is repeating can be any string of outcomes repeated over and over. However, the most common repeating pattern is a sequence of n taken outcomes followed by a single not-taken outcome, followed by n taken outcomes and so on. A representative example of this type of pattern is **110 110 110**, where a **1** represents a taken outcome and a **0** represents a not-taken outcome. This is the execution pattern we would expect to see from the loop ending branch of a for-loop. We measured the frequency of repeating patterns and found that this type of repeating pattern represents 18% of all the branches in the SPECint95 benchmarks. The inverse of the pattern (e.g., **001 001 001**) accounts for 4% of all branches. All other repeating patterns combined account for 4% of all branches. This includes all repeating patterns, regardless of

whether they were caused by a loop-branch. More information about repeating patterns can be found in [6].

There are also branches that behave systematically, but not in repeating patterns. Consider the task of parsing through any type of nonrandom input. A branch that is making decisions based on each character will see the same sequences over and over again. If you read through this paragraph and look for words starting with “ove” you will realize that it is always followed by the letter “r.” In the same way as written text, branches based on input data will often follow this type of behavior. In a previous study [7], it was indicated that approximately 30% of all branches have some amount of systematic behavior not accounted for by repeating patterns or branch bias. The go and xisp benchmarks had the largest proportions of such systematic behavior. The strength and nature of this systematic behavior has to our knowledge not been studied in depth.

C. Correlation Between Branches

The outcomes of branches that are close together in the source code are often dependent or related. This is because they are often based on the same or related variables. Knowing the outcome of the earlier branch therefore often helps us predict the outcome of the later branch.

Two branches are said to be correlated if there is a reciprocal relation between their outcomes. That is, if knowing the outcome of one branch provides information about the likely outcome of the other branch, the two branches are correlated. The correlation may be strong, such that the outcome of the first branch can be confidently used to predict the second branch, or the correlation may be weak, such that the outcome of the first branch only hints at what the second branch will do. Although strong correlation is more effective for prediction, weak correlation can also be useful for hard-to-predict branches.

There are two obvious reasons for the directions of two branches to be correlated. One is that the conditions of the two branches are based (fully or partly) on the same or related information. An example of this is shown in Fig. 5 (each of the if statements is turned into a branch by the compiler). In this example, the second condition, (cond1 & cond2), can only be true if the first condition, (cond1), is true. When the

```

if (cond1)
    index++;
if (cond1 & cond2)
    ...

```

Fig. 5. Correlation example 1.

```

if (cond1)
    a = 2;
if (a == 0)
    ...

```

Fig. 6. Correlation example 2.

if statements are turned into branches by the compiler, these branches will be correlated. The other obvious reason for two branches to be correlated is that information affecting the outcome of the current branch is generated based on the outcome of the first branch. In the example in Fig. 6, we can see that if the first condition is true, the variable *a* is set to the value 2 and the second condition will therefore be false. Once again, the branches created by the compiler will be correlated.

Correlation between branches is a very important type of branch behavior and several of the best predictors are based on this effect. Previous studies [6] have shown that over 60% of all branches have some correlation with the outcomes of recently executed branches. Often, the next outcome of a branch can be predicted equally well using correlation based prediction, or by detecting the repeating or systematic behavior described above.

IV. AN OVERVIEW OF CURRENT PREDICTION SCHEMES

In this section, we describe several existing branch prediction schemes and tie them back to the branch behavior that they capture. We also provide simulation results showing their relative performance. The results are often sensitive to the benchmarks used, but the evaluation should give a good indication of the type of branch predictors that are likely to be useful. Due to space limitations, we do not cover all existing predictors. However, the predictors examined here should give a good overview of the fundamental mechanisms available.

A. Basic Branch Predictors

The branch prediction algorithms that are easiest to implement are those requiring no history, or only a single level of history. These algorithms mainly focus on capturing the dominant direction of a branch. The algorithms evaluated here are (with the labels used on the graph in parenthesis).

- Always Predict Taken (taken). Since a taken outcome is slightly more likely than a not-taken outcome, this can be used as a very simple predictor.
- Backward Taken, Forward Not-Taken (BTFN). If the branch displacement is positive, predict not-taken. Otherwise predict taken. The basis for this algorithm is that backward branches are often loops which are mostly taken. The compiler can then arrange the remaining branches such that the most likely path is not-taken.

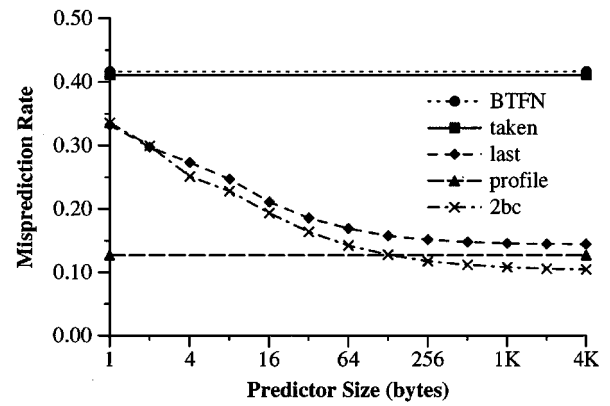


Fig. 7. Accuracy of basic branch predictors.

- Simple Profiling (profile) [8]. The code is profiled using separate profiling input to determine the most frequent direction for each branch. This direction is encoded in the branch instruction and used as the prediction during the final run of the program.
- Last-Time (last) [9]. The address of the branch is used to index into an array of one bit elements to find the prediction for that branch. If the bit is one the branch is predicted taken. Otherwise, it is predicted not-taken. When the branch is committed, the bit is updated based on the actual direction of the branch.
- Two-Bit Counter (2bc) [9]. Similar to the Last-Time predictor, but each element is a two-bit saturating counter. The most significant bit of the counter is used to predict the branch. The counter is updated when the branch is committed. If the branch is taken, the counter is incremented. Otherwise it is decremented. The two-bit mechanism is able to ignore a single divergent outcome and, therefore, works better than the Last-Time predictor for loops and similar branches.

Fig. 7 shows the average misprediction rate for the SPECint95 benchmarks for each of these predictors. The vertical axis shows the misprediction rate and the horizontal axis shows the size of the predictor. The size is calculated as the storage space used for dynamic history information, such as counters. The static predictors do not need storage for dynamic history, so their performance is represented by horizontal lines.

The Always Taken and Backward Taken, Forward Not-Taken predictors have high misprediction rates—41.1% and 41.6%, respectively. The accuracy of the BTFN predictor can be improved using compiler support to make not-taken the most likely outcome of forward branches. These two predictors are cheap, require little hardware support and represent an improvement over using no prediction at all. However, they are not appropriate for modern microprocessors. The misprediction rate of the other static predictor, Simple Profiling, is 12.7%. While this is much better than the other static predictors, one out of every eight branches is still mispredicted and the program must be profiled when it is compiled.

The dynamic predictors learn the behavior of the branches during the run of the program and can adapt to changes

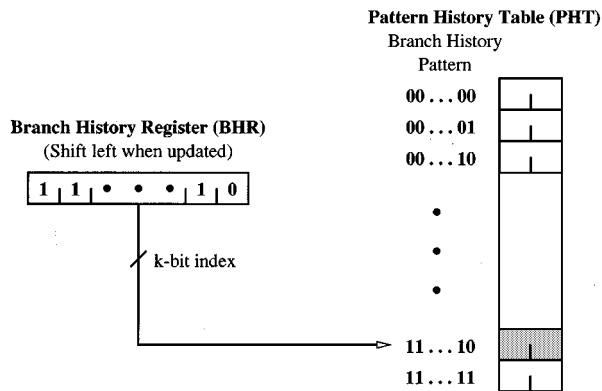


Fig. 8. Diagram of a GAs two-level predictor.

during execution. They can therefore achieve lower misprediction rates without profiling. As the prediction tables of the dynamic predictors increase in size, there is less conflict between different branches using the table. This leads to lower misprediction rates as the size increases. For these simple dynamic predictors, most of the performance can be achieved with a table of approximately 256 bytes and there are only very marginal gains from increasing the size beyond 2 kB. At 2 kB, the misprediction rates of the Last-Time and Two-Bit Counter predictors are 14.5% and 10.5%, respectively. To improve branch prediction beyond this, we must consider more history information.

B. Two-Level Adaptive Branch Predictors

Even the best of the simple prediction schemes, a table of two-bit saturating counters, mispredicted 10.5% of all branches. More accurate predictions can be made by using two levels of history. Whereas the Last-Time and Two-Bit Counter schemes both try to predict the dominant direction of a branch, the Two-Level Adaptive [10], [11] predictor collects a first level history of execution outcomes and then predicts the direction separately for each of the possible patterns in the first level history. The first level history is recorded in one or more k -bit shift registers, called branch history registers (BHRs), which record the outcomes of the k most recent branches. The second level history is recorded in one or more tables, called pattern history tables (PHTs), of two-bit saturating counters. The BHR is used to index into the PHT to select which two-bit counter to use. If the configuration uses more than one PHT, one of the PHTs is chosen based on the least significant bits of the fetch address. Once the two-bit counter is selected, the prediction is made using the same method as in the Two-Bit Counter scheme.

A global two-level predictor (GAs) uses one branch history register to collect the outcomes of all branches. Fig. 8 shows a GAs predictor with one PHT. The contents of the BHR are used to index into the PHT. This selects a two-bit counter, shaded in the figure. The prediction for the current branch is based on the most significant bit of this counter as explained earlier. The BHR is updated by shifting left by one, with the prediction that was just made entered as the least significant bit. The two-bit counter that was used to make the

Table 2

Configurations Used for Two-Level Adaptive Branch Predictors. The Configurations Are Represented as (HL, PHTs) Where HL Is the History Length in Bits and PHTs Is the Number of Pattern History Tables

Size	Predictor		
	GAs (HL,PHTs)	Gshare (HL,PHTs)	PAs (HL,PHTs)
1 KB	(6,64)	(6,64)	(3,128)
4 KB	(9,32)	(8,64)	(8,32)
16 KB	(12,16)	(12,16)	(16,1)
64 KB	(14,16)	(14,16)	(18,1)
256 KB	(16,16)	(20,1)	(20,1)

prediction is updated when the branch is committed using the same rules as for the Two-Bit Counter scheme. Since the outcomes in the BHR represent the branches preceding the current branch, the global two-level predictor can use the correlation between the current branch and the other branches in the history to make its prediction. A variation (Gshare) of the GAs predictor XORS the fetch address with the BHR to create the index into the PHT. This achieves better utilization. For further details on the implementation of GAs and Gshare, see [11], [12]. There are variations of the global two-level predictor that use the path defined by branch target addresses instead of the history pattern [13], [14]. Other variations of the global two-level predictor detect the best history length to use for a given program or branch either dynamically or using profiling [14], [15] to achieve higher accuracies. These path-based and variable history length schemes are not evaluated here.

A per-address two-level predictor (PAs) uses one branch history register (BHR) per branch, stored in a branch history table (BHT). Each of the history registers collects the outcomes of one branch. The BHT is a tagged structure similar to a branch target buffer (BTB). To make a prediction, the BHR corresponding to the current branch is selected and this history is used to make a prediction in the same way as in the GAs predictor.¹ Since the PAs predictor uses the previous execution pattern of the branch to make a prediction, it is able to correctly predict branches with repeating or systematic execution patterns. For further details on the implementation of the PAs predictor, please see [3], [10]. The two-level predictors represent a great improvement over previous schemes. However, they need larger structures to reach their full potential and they take longer to train than simpler predictors.

We evaluated the performance of these three two-level predictors, simulating all possible configurations for each one. The configurations that yielded the lowest misprediction rate for each predictor are listed in Table 2. For the PAs predictor, the cost of the branch histories in the BHT was included, but the cost of the tags was not as it is assumed that these are shared with the BTB or a similar structure. If no configuration was exactly the desired size, the closest size was used. A 2-k entry four-way set-associative BHT was used for PAs,

¹To reduce the access time of the predictor, it is common to run the prediction one branch execution ahead. This eliminates the need for sequential table accesses.

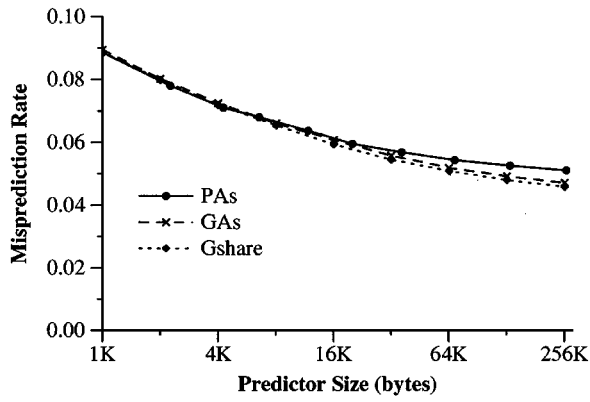


Fig. 9. Comparison of two-level predictors.

Table 3
Misprediction Rates of 64-kB Predictors

Benchmark	Gshare	PAs
compress	8.23%	5.38%
gcc	5.09%	7.76%
go	10.97%	17.43%
jpeg	6.99%	4.23%
m88ksim	1.59%	1.31%
perl	2.18%	2.84%
vortex	0.86%	1.10%
xlisp	4.81%	3.50%

as this proved to be a robust choice for a variety of predictor sizes. The configurations given in the table perform best on average for the benchmarks, but there is some variation between the individual benchmarks. As a general rule, the optimal configuration for larger benchmarks has more PHTs whereas for smaller benchmarks a longer history is better.

The three two-level predictors are compared for sizes ranging from 1 to 256 kB in Fig. 9. Gshare, GAs, and PAs achieve nearly identical misprediction rates for predictors smaller than 8 kB. PAs uses a different type of information to make predictions than GAs and Gshare, but each type of information is equally useful on average. For larger predictors, PAs performs worse than Gshare and GAs. Gshare is marginally better than GAs at all sizes. GAs and Gshare exploit the same type of correlation for prediction, but the hashing function in Gshare is better. For comparison, if the Two-Bit Counter predictor was included in the figure, its misprediction rate of 10.5% would be off the scale.

Even for 64-kB predictors, the accuracy of Gshare, GAs, and PAs is close. At this size, Gshare has a misprediction rate of 5.09%, GAs 5.19%, and PAs 5.44%. This indicates that Gshare is likely to be the best of these predictors at this size. However, even though Gshare is on average better than PAs, there is large variation between the benchmarks. Table 3 shows the misprediction rates of the Gshare and PAs predictors for the individual benchmarks. Gshare is substantially better than PAs for go, gcc, and perl. PAs is substantially better than Gshare for compress, jpeg, and xlisp. We will see later that hybrid branch predictors, which use both a Gshare and a PAs predictor, do better than each of those predictors alone.

C. PHT Interference and Interference Reduction Schemes

One of the major limitations of the two-level branch predictors is that the branches in a program compete for the same counters in the second-level table, the PHT. Each branch may be represented once for each possible history pattern, so in a finite size table, we are likely to have several collisions where two branches want to use the same counter. For modestly sized two-level predictors, as much as one in three mispredictions can be due to interference. We briefly examined the impact of interference on Gshare and PAs by simulating three different size predictors. We also simulated the corresponding predictors using the same amount of history, but having one PHT per branch and, therefore, no interference.² The base configuration has only a single PHT.

If all interference were removed, the average misprediction rate of a 4-kB Gshare predictor would be 4.61% instead of 7.19%. For a 16-kB Gshare, the misprediction rate of the interference free predictor is 4.42% instead of 5.94%, and for the 64-kB predictor, the misprediction rate is 4.30% instead of 5.09%.

If all interference were removed, the average misprediction rate of a 5-kB PAs predictor would be 5.48% instead of 7.11%. For a 20-kB PAs, the misprediction rate of the interference free predictor is 5.10% instead of 5.96%, and for the 68-kB predictor, the misprediction rate is 4.95% instead of 5.44%. As is the case with Gshare, the negative effects of interference diminish for larger predictor sizes. Most of the work on interference has targeted global two-level predictors, but as we can see, per-address two-level predictors are also significantly impacted by interference.

Several mechanisms have been proposed to improve global variations of the two-level predictors by reducing the amount of interference in the PHTs. Branch Filtering [16] uses a mechanism to dynamically identify strongly biased branches and predict these using a simple Last-Time predictor. The two-level predictor is only used for the less biased branches, reducing the interference between branches in the PHTs. The Agree predictor [17] makes a preliminary prediction for each branch the first time the branch is seen. Subsequently the two-level predictor is used to predict whether to agree or disagree with that prediction. Since the original prediction will be correct the majority of the time, the agree outcome will be dominant, thus making the interference less destructive. The Skewed [18] branch predictor uses three predictor banks, or PHTs, each indexed with slightly different hash functions. The prediction of the Skewed branch predictor is the prediction made by two or more of the banks; therefore, it is only hurt by interference if it occurs in two banks simultaneously. The Bi-Mode [19] predictor uses a table of two-bit counters indexed by the branch address to make an initial prediction. This initial prediction is used to select which of two Gshare predictors, which were accessed in parallel with the table, to use for the prediction. Only the state of the selected Gshare predictor is updated when the branch resolves. These four predictors

²We cannot build the interference free predictors, as they require one PHT per branch and would therefore be prohibitively large.

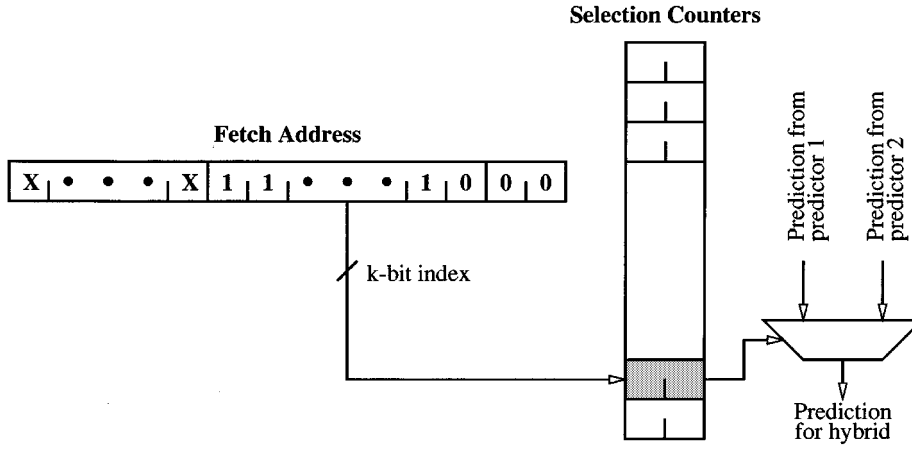


Fig. 10. Diagram of a hybrid branch predictor.

were compared in [20] and it was found that depending on the assumptions, Branch Filtering or the Bi-Mode predictor would outperform the other mechanisms.

D. Hybrid Branch Predictors

Due to the many different ways a branch can behave, a single predictor may not be the best for all branches. This is the motivation for building hybrid branch predictors. A hybrid branch predictor consists of two or more component predictors and a selection mechanism to choose which of the predictors to use for each branch. Fig. 10 illustrates how a hybrid predictor generates a prediction. The selection mechanism on the left uses a table of two-bit saturating counters to keep track of which predictor is currently more accurate for each branch. Each branch is mapped to a counter using the least significant bits in its fetch address. If the counter value is larger than or equal to 2, the first predictor is used. Otherwise, the second predictor is used. The counter is updated when the branch is committed. If only the first predictor was correct, the counter is incremented. If only the second predictor was correct, the counter is decremented. If both predictors made the same prediction, the counter value is not changed. The component predictors used in the hybrid can be any predictors, but it is generally best to have the components complement each other.

In this section, two such hybrid branch predictors are evaluated. Both of them use Gshare as one component, with the other component being either PAs or the Two-Bit Counter predictor. These schemes are referred to as the PAs/Gshare and 2bc/Gshare predictors. The selection mechanism is a table of saturating counters, such as the one in [12]. However, the selection mechanism uses three-bit saturating counters instead of two-bit counters, as we found these to work better.

It is not trivial to come up with the best configurations for hybrid branch predictors. Each of the component predictors have their own parameters and the size of each component and the selection mechanism are other parameters. For this experiment, the configurations were based on the original paper [12] on hybrid predictors. For the PAs/Gshare predictor, the hardware is shared almost equally between the selection mechanism, PAs, and Gshare. For the 2bc/Gshare

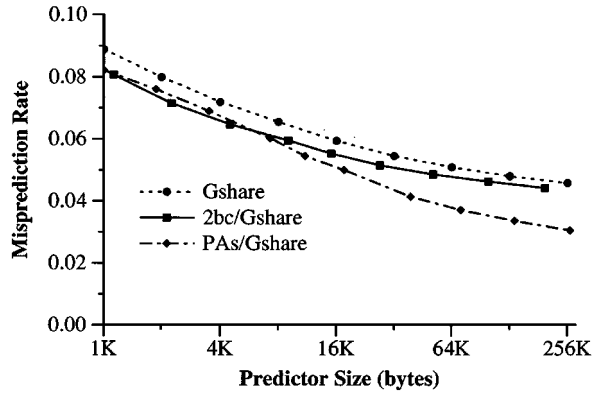


Fig. 11. Accuracy of hybrid branch predictors.

predictor, half of the hardware is allocated to the Gshare component and approximately a quarter each to the Two-Bit Counter predictor and the selection mechanism. In both cases, selection mechanisms over 3 kB were not considered as this is unlikely to improve performance. For each of the component predictors, the configurations with only a single PHT were used. This was used even though it may not be optimal for the smallest predictors.

Fig. 11 shows the performance of the two hybrid predictors along with a Gshare predictor for comparison. For predictors smaller than 4 kB, the 2bc/Gshare hybrid is slightly better than the PAs/Gshare hybrid. Over 8 kB, PAs/Gshare is better, with a large performance advantage for sizes over 32 kB. At an approximate cost of 64 kB, PAs/Gshare has a misprediction rate of 3.7%, 2bc/Gshare has a misprediction rate of 4.8%, and Gshare has a misprediction rate of 5.1%. In addition to the hybrid predictors shown here, researchers have suggested two-level selection mechanisms [21], hybrids with more than two components [22] and hybrids with static selection mechanisms [23].

E. Summary of Branch Predictor Performance

This section has provided a general evaluation of many of the important branch prediction schemes available today. This is not an exhaustive study of all available schemes, but a representative mix of the predictors that are available. A

number of different strategies in branch prediction were evaluated: static predictors, simple dynamic predictors, two-level predictors, and hybrid branch predictors.

In summary, Simple Profiling can achieve a misprediction rate of 13% without dynamic branch prediction hardware. The Two-Bit Counter scheme is a simple dynamic predictor that has a misprediction rate of just over 10% at low complexity and cost. Two-level predictors require slightly more complex control logic, but provide lower misprediction rates at low cost and approach 5% for 64-kB predictors. Hybrid branch predictors achieved even lower misprediction rates, under 4% for 64-kB predictors and promise accuracy growth well beyond the sizes that were investigated here.

V. DESIGNING PREDICTORS FOR HIGH-PERFORMANCE MICROPROCESSORS

In this section, we discuss some of the problems facing the designer of future high-performance microprocessors and look at some possible solutions. The best solution in each case will depend on a number of issues such as process technology, pipeline depth, and processor microarchitecture. For all of the solutions that are listed, we also explain the advantages and disadvantages of that solution.

A. Problems

There are three parts to making a branch prediction: Determining that a branch is being fetched, predicting the direction of the branch, and predicting the target of the branch. Initially, it may sound like a good idea to set the size and complexity of the predictor to what would fit in one cycle. We argue that this is probably not the case. To make a true single-cycle prediction, we must in parallel make a direction prediction, a target prediction, determine the not-taken branch address, and decide whether we are fetching a branch this cycle. If we are fetching a branch, the direction prediction is used to select between the target address and the not-taken address, otherwise we use the not-taken address. Finally, this address is fed back into the branch predictor in the next cycle.

Predicting the direction of the branch generally consists of using the address and some additional information to look up a prediction in a table of counters. According to [24], future processors will likely only be able to access tables up to 0.125 or 8 kB in a single cycle, depending on the process technology and aggressiveness of the frequency. This does not include the delay of complex selection logic, etc. It is therefore clear that larger predictors and hybrid predictors will have a prediction latency of more than one cycle if current design trends hold.

Predicting the target of a branch is also hard to fit in a single cycle. Depending on the implementation, a branch target buffer (BTB) is likely to hold somewhere between 4 and 8 bytes per entry, including tags, target address, and other branch information. A 256 entry BTB, for example, is 1–2 kB in size and will still have a significant miss rate. In addition, the BTB needs to be associative for best performance adding

the need for a way-mux on the critical path of determining the target address. Once again, we see that only the smallest branch target buffers are likely to fit within a cycle.

Determining whether there is a branch in the current fetch can be done using the BTB with the same access time problems. It can also be done using predecode bits coming from the I-cache, or by examining the I-stream coming from the I-cache. However, anything using information from the I-cache is likely to take as long as or longer than the BTB. If we are willing to use a separate structure to determine whether there is a branch in the current fetch, this latency can be overcome.

Finally, when we have the target address, not-taken address, and direction prediction, we still must contend with the next fetch PC mux. The next fetch PC mux will usually have a large number of addresses to choose from. Some of the most common candidates are taken and not-taken address, target address of mispredicted branch, exception redirect address, decode detected branch target, return address stack prediction, and indirect jump target prediction. To exacerbate this problem, most of these inputs are equally important and time critical. The single-cycle prediction path can be favored only at the cost of slowing the other paths down. The next fetch PC mux is an overhead that takes time away from the branch predictor.

Although it in many ways sounds appealing to limit the predictor size and complexity to what can fit in one cycle, we see that this would be extremely limiting. It would significantly reduce accuracy and lead to poor performance. We therefore examine other options.

B. Updating a Two-Level Predictor

Before examining the other implementation options, we take a quick look at how the update for a two-level predictor works. Two different histories must be updated for each branch: the history register (global or per-address) and the pattern history table.

The history register must be updated speculatively in order to have the proper context for predicting the following branches. This means that on a branch misprediction there will be some incorrect outcomes recorded in the history. Therefore, a checkpointing (or similar) mechanism must be used to recover the original contents of the history on a branch misprediction.

Unlike the history register, a speculative update to the PHT would not help future predictions. The PHT entry is a saturating counter, so the speculative update could strengthen the prediction but not change it. Only a misprediction can change the prediction of a PHT entry. The fact that a branch is mispredicted is not known until execute time. Therefore, the PHT should be updated either at execute or commit time. If the PHT is updated at execute time, there is a small pollution effect due to branches executed down the wrong path. However, if the PHT is updated at commit time, mispredicted branches must be handled as a special case. If a mispredicted branch would have flipped the prediction of a counter, it is important that the new prediction is available to subsequent fetches down the correct path.

C. Multicycle Prediction

Pipelining the predictor to take two or three cycles allows time for larger or more complex predictors. As long as the branch predictor does not predict a taken branch, the fetch engine continues to fetch sequential cache blocks (regardless of the number of branches that are in that block). The address of the sequential cache block can be determined using an adder. If the branch predictor predicts a taken branch, the fetch is redirected and we pay the cost of a one- or two-cycle taken branch bubble.

There are a few issues that this mechanism brings up. We must predict for the entire cache line up to the first taken branch at a time. Also, history about the not-taken branches fetched in the last cycle or two is only available for the last cycle of the prediction unless a special structure is used to get this information earlier. However, this information can still be used as a late select. Finally, the taken branch bubble creates empty fetches so some buffering may be needed to have consistent fetch bandwidth.

D. Instruction Buffering

Whenever we create bubbles in the fetch pipeline, we potentially get underutilization of the rest of the processor. However, if the fetch pipeline has a higher sustained bandwidth than the rest of the machine, we can remove this problem using an instruction buffer after fetch or decode. An instruction buffer can help us give consistent bandwidth to the rest of the processor. It also decouples the fetch engine so that we can have significant runahead fetching if desired.

The disadvantages with using instruction buffering are that the structure itself may be complex and that it is likely to add latency to the front-end pipeline.

E. Simple-Single Cycle Prediction With Multicycle Override

To avoid the taken branch bubble, we can modify a multicycle predictor slightly. Instead of always predicting not-taken as the initial prediction, we can use a very simple predictor.

The simple predictor can be anything that will fit in a single cycle. The Alpha 21264 [25] uses a Last-Time next cache index and way predictor for the single-cycle prediction. Alternatively, we could use a small Branch Target Buffer along with a Two-Bit Counter predictor of limited size.

When the multicycle prediction is made later, it is compared to the single-cycle prediction. If the two predictions were the same, no action is needed. Otherwise, the fetch engine is resteered to the address predicted by the more accurate multicycle prediction. In addition to eliminating the taken branch bubble, this also eliminates the problem of having the most recent branch history unknown until the last stage of the multicycle prediction.

The disadvantages of this scheme is that it adds complexity compared to the multicycle prediction. The next PC mux gets extra time-critical inputs and we add a comparison (one XOR gate) to the prediction path for the override.

F. Lookahead Prediction

The problem with the previous schemes is that the latency for the most accurate prediction is two or more cycles. This means that whenever the simple predictor is not correct, we take a penalty.

The penalty can be avoided by predicting two fetches ahead. Instead of predicting the branch(es) that are fetched this cycle, you predict the branches that will be fetched next cycle. A two-cycle prediction will now be available for resteering fetch as the predicted branch is being fetched. One example of a lookahead scheme was published in [26].

The disadvantages of most lookahead schemes is that there is a large degree of extra complexity added to the prediction. Especially the BTB design and the misprediction recovery is complicated using this method. Lookahead schemes are usually a little less accurate than similar prediction algorithms without lookahead. It is possible to use a lookahead predictor as a first prediction, with a normal multicycle predictor that can override it with an even more confident prediction.

G. Predicting the Target Address

A target address is needed whenever a branch is predicted to be taken. The above strategies also apply to target address prediction. However, there are some additional issues with target prediction.

It is harder to do accurate lookahead prediction for the branch target than it is for the branch direction. This is because the direction predictor has much smaller elements (two-bit counters) rather than 32 or 64-bit addresses. For a lookahead BTB, we need to store the addresses of targets down several possible paths and then select which target address to use based on the prediction of the previous cycle. This is costly, but depending on the situation may still be worthwhile.

Due to the number of bits in the address, comparing an initial prediction to a more advanced prediction takes longer for the target address than for the direction prediction. This may increase the latency of schemes with multicycle override by one cycle. However, the increased overriding latency will still be much smaller than the latency of a full misprediction, so this remains an attractive option.

To reduce the size of the BTB, it is possible to only store the predicted cache index and I-cache way of the target. This mechanism can be used with both lookahead prediction and multicycle override, but it does bring up several issues. The most important one is that the full instruction cache and BTB hit indication as well as the TLB translation is delayed. Storing only the cache set and way in the BTB may make sense if it creates a single-cycle turnaround. However, as is always the case with computer architecture the decision will be made based on tradeoffs in the overall fetch engine design.

VI. CONCLUSION

In this paper, we discussed several aspects of branches and branch prediction. We showed that for the majority of branches, the behavior is well understood. However, for some of the hard to predict branches we know less about the behavior.

A number of previously proposed branch predictors were explained and evaluated. The misprediction rates varied from over 40% for the worst static predictors to less than 4% for the best and costliest, of the dynamic predictors.

Finally, we identified some of the problems in future branch prediction design. We presented possible approaches to address these problems, but also showed that all of these approaches have some disadvantages relative to the ideal case. The predictor in a high-frequency future microprocessor design is unlikely to reach the performance of the ideal predictor design. However, with clever microarchitectural work it could get close.

REFERENCES

- [1] B. Calder, D. Grunwald, and J. Emer, "A system level perspective on branch architecture performance," in *28th ACM/IEEE Int. Symp. Microarchitecture*, 1995.
- [2] Welcome to SPEC. The Standard Performance Evaluation Corporation. [Online]. Available: <http://www.specbench.org/>
- [3] T.-Y. Yeh, "Two-level adaptive branch prediction and instruction fetch mechanisms for high performance superscalar processors," Ph.D. dissertation, Univ. Michigan, Ann Arbor, 1993.
- [4] D. Kaeli and P. Emma, "Branch history table predictions of moving target branches due to subroutine returns," in *18th Annu. Int. Symp. Computer Architecture*, 1991.
- [5] P.-Y. Chang, E. Hao, and Y. N. Patt, "Predicting indirect jumps using a target cache," in *Proc. 24th Annu. Int. Symp. Computer Architecture*, 1997.
- [6] M. Evers, "Improving branch prediction by understanding branch behavior," Ph.D. dissertation, Univ. Michigan, Ann Arbor, 2000.
- [7] M. Evers, S. J. Patel, R. S. Chappell, and Y. N. Patt, "An analysis of correlation and predictability: What makes two-level branch predictors work," in *25th Annu. Int. Symp. Computer Architecture*, 1998.
- [8] J. A. Fisher and S. M. Freudenberger, "Predicting conditional branch directions from previous runs of a program," in *5th Int. Conf. Architectural Support for Programming Languages and Operating Systems*, 1992.
- [9] J. E. Smith, "A study of branch prediction strategies," in *8th Int. Symp. Computer Architecture*, 1981.
- [10] T.-Y. Yeh and Y. N. Patt, "Two-level adaptive branch prediction," in *24th ACM/IEEE Int. Symp. Microarchitecture*, 1991.
- [11] —, "Alternative implementations of two-level adaptive branch prediction," in *19th Annu. Int. Symp. Computer Architecture*, 1992.
- [12] S. McFarling, "Combining branch predictors," Digital Equipment Corporation, WRL Tech. Note TN-36, 1993.
- [13] R. Nair, "Dynamic path-based branch correlation," in *28th ACM/IEEE Int. Symp. Microarchitecture*, 1995.
- [14] J. Stark, M. Evers, and Y. N. Patt, "Variable length path branch prediction," in *Proc. 8th Int. Conf. Architectural Support for Programming Languages and Operating Systems*, 1998.
- [15] T. Juan, S. Sanjeevan, and J. J. Navarro, "Dynamic history-length fitting: A third level of adaptivity for branch prediction," in *25th Annu. Int. Symp. Computer Architecture*, 1998.
- [16] P.-Y. Chang, M. Evers, and Y. N. Patt, "Improving branch prediction accuracy by reducing pattern history table interference," in *Proc. Int. Conf. Parallel Architectures and Compilation Techniques*, 1996.
- [17] E. Sprangle, R. S. Chappell, M. Alsup, and Y. N. Patt, "The agree predictor: A mechanism for reducing negative branch history interference," in *Proc. 24th Annu. Int. Symp. Computer Architecture*, 1997.
- [18] P. Michaud, A. Seznec, and R. Uhlig, "Trading conflict and capacity aliasing in conditional branch predictors," in *24th Annu. Int. Symp. Computer Architecture*, 1997.
- [19] C.-C. Lee, I.-C. K. Chen, and T. N. Mudge, "The bi-mode branch predictor," in *30th ACM/IEEE Int. Symp. Microarchitecture*, 1997.
- [20] C.-C. Lee, "Optimizing high performance dynamic branch predictors," Ph.D. dissertation, Univ. Michigan, Ann Arbor, 1997.
- [21] P.-Y. Chang, E. Hao, and Y. N. Patt, "Alternative implementations of hybrid branch predictors," in *28th ACM/IEEE Int. Symp. Microarchitecture*, Nov. 1995.
- [22] M. Evers, P.-Y. Chang, and Y. N. Patt, "Using hybrid branch predictors to improve branch prediction accuracy in the presence of context switches," in *23rd Annu. Int. Symp. Computer Architecture*, 1996.
- [23] D. Grunwald, D. Lindsay, and B. Zorn, "Static methods in hybrid branch prediction," in *Proc. 1998 ACM/IEEE Conf. Parallel Architectures and Compilation Techniques*, 1998.
- [24] D. A. Jimenez, S. W. Keckler, and L. Calvin, "The impact of delay on the design of branch predictors," in *33rd ACM/IEEE Int. Symp. Microarchitecture*, Dec. 2000.
- [25] L. Gwennap, "Digital 21 264 sets new standard," *Microprocessor Rep.*, 1996.
- [26] A. Seznec, S. Jourdan, P. Sainrat, and P. Michaud, "Multiple-block ahead branch predictors," in *Proc. 7th Int. Conf. Architectural Support for Programming Languages and Operating Systems*, 1996.



Marius Evers received the Ph.D. degree from the University of Michigan.

He is a Member of Technical Staff at Advanced Micro Devices, Sunnyvale, CA. He has done extensive research in branch prediction, focusing on understanding the behavior of branches. He is the author or co-author of ten papers.



Tse-Yu Yeh (Member, IEEE) received the B.S. degree from the National Taiwan University and the Ph.D. degree from the University of Michigan.

He is an Architect and Principal Scientist at the Broadband Processor Business Unit of Broadcom Corporation (formerly the SiByte Corporation), Santa Clara, CA. He is the Technical Leader and Manager in the architecture and verification of a MIPS-64 processor core development. Prior to joining SiByte, he worked at Intel where he was

a key member in the IA-64 instruction set definition and the microarchitecture design of the Itanium processor. He has published many papers and filed many patents in the areas of branch prediction, instruction fetch and microprocessor design. While at the University of Michigan, he and his Ph.D. advisor, Prof. Yale Patt, co-invented the Two-Level Adaptive Branch Prediction, which has been used in many modern microprocessor designs.